

CHAPTER 7: INPUT/OUTPUT SYSTEMS

INPUT/OUTPUT ORGANIZATION

- This chapter focuses on the methods and components used to **move data** between the **CPU/Memory** and the **outside world** (peripherals).
- It addresses the challenges of communicating with a wide variety of devices that are much slower than the CPU.

I/O SPEED AND COORDINATION ISSUES

The central problem of I/O is a **mismatch** in properties:

- **Speed Mismatch:** Peripherals (like keyboards, hard drives) are **orders of magnitude slower** than the CPU and RAM. The CPU can't just sit and wait for them.
- **Data Format Mismatch:** Devices may use different data formats, word lengths, or signal types than the CPU.
- **Coordination:** How does the CPU know when a device is ready to send data (e.g., a key press) or has finished receiving data (e.g., a file write)?

I/O DEVICE INTERFACES

- An interface is the boundary where a device connects to the system. It has two parts:
 1. **Port:** The physical connector (e.g., USB port, HDMI port).
 2. **Interface Standard:** The electrical and logical rules for data transfer (e.g., the USB 3.0 protocol, the SATA standard).

I/O MODULE FUNCTIONS

An **I/O Module** (also called a device controller or adapter) is the "middle-man" hardware that connects a device to the system bus. Its key functions are:

- **Control & Timing:** Coordinates data transfer to/from the device, respecting its speed.
- **CPU Communication:** Provides a way for the CPU to send commands (e.g., "read block 5") and receive status updates (e.g., "data ready").

- **Device Communication:** Sends commands and receives data from the specific peripheral.
- **Data Buffering:** Uses small, on-board memory (a **buffer**) to hold data temporarily, smoothing out the speed mismatch.
- **Error Detection:** May check for transmission errors (e.g., parity bits).

CPU INTERRUPTS (DIAGRAM OF INTERRUPT CYCLE)

An **interrupt** is a hardware signal from an I/O module to the CPU, indicating that an event needs immediate attention (e.g., "I/O complete").

THE INTERRUPT CYCLE:

1. **Device Issues Interrupt:** The I/O module sends an interrupt signal to the CPU.
2. **CPU Finishes Instruction:** The CPU completes its *current* instruction.
3. **CPU Acknowledges & Saves State:** The CPU saves its current state (Program Counter, registers) onto the system **stack**.
4. **CPU Loads ISR Address:** The CPU identifies the interrupting device and jumps to the address of its specific **Interrupt Service Routine (ISR)**.
5. **ISR Executes:** The ISR (a special piece of code) runs to handle the I/O event (e.g., read data from the device's buffer).
6. **CPU Restores State:** After the ISR finishes, the CPU restores its original state from the stack.
7. **CPU Resumes:** The CPU continues executing the original program as if nothing happened.

DIRECT MEMORY ACCESS (DMA)

- **Problem with Interrupts:** For large data transfers (like disk reads), using interrupts for *every single byte* (known as Programmed I/O) is very inefficient and bogs down the CPU.
- **Solution: DMA.** A special **DMA Controller** is given permission to transfer data *directly* between the **I/O module** and **Main Memory** *without* involving the CPU.
- **Process:**

1. The CPU sets up the DMA transfer (e.g., "move 10KB from device X to memory address Y").
2. The CPU goes back to its other work.
3. The DMA controller manages the entire transfer, "stealing" memory bus cycles from the CPU when needed.
4. When the *entire block* transfer is finished, the DMA controller sends a *single* interrupt to the CPU.

EXTERNAL BUS AND PORT INTERFACES

- **Bus:** A shared communication pathway (e.g., **PCIe bus**) used to connect internal components like the graphics card, network card, and I/O controllers.
- **Port:** The external connection point for peripherals.
 - **Serial Port:** Transmits data one bit at a time (e.g., USB, Ethernet).
 - **Parallel Port:** Transmits multiple bits at once (e.g., old printer ports).

OS I/O SUBSYSTEM

- The **Operating System** (OS) provides a software layer to manage I/O, hiding the hardware complexity from applications.
- **Key Components:**
 - **Device Drivers:** Hardware-specific software that knows how to "talk" to a particular I/O module.
 - **Standard API:** Provides applications with simple commands (e.g., `open()`, `read()`, `write()`) that work for all devices.
 - **Buffering/Caching:** Manages memory buffers to improve performance.
 - **Spooling:** Queues I/O requests for devices that can't handle multiple requests at once (e.g., a printer).

PERFORMANCE CONSIDERATIONS

- **Latency (Response Time):** The time it takes to *start* an I/O operation (e.g., disk seek time).

- **Throughput (Bandwidth):** The rate at which data can be transferred *after* the operation starts (e.g., MB/s).
- **Bottlenecks:** The goal is to identify and remove the slowest part of the I/O path, which may be the device, the bus, or the software.